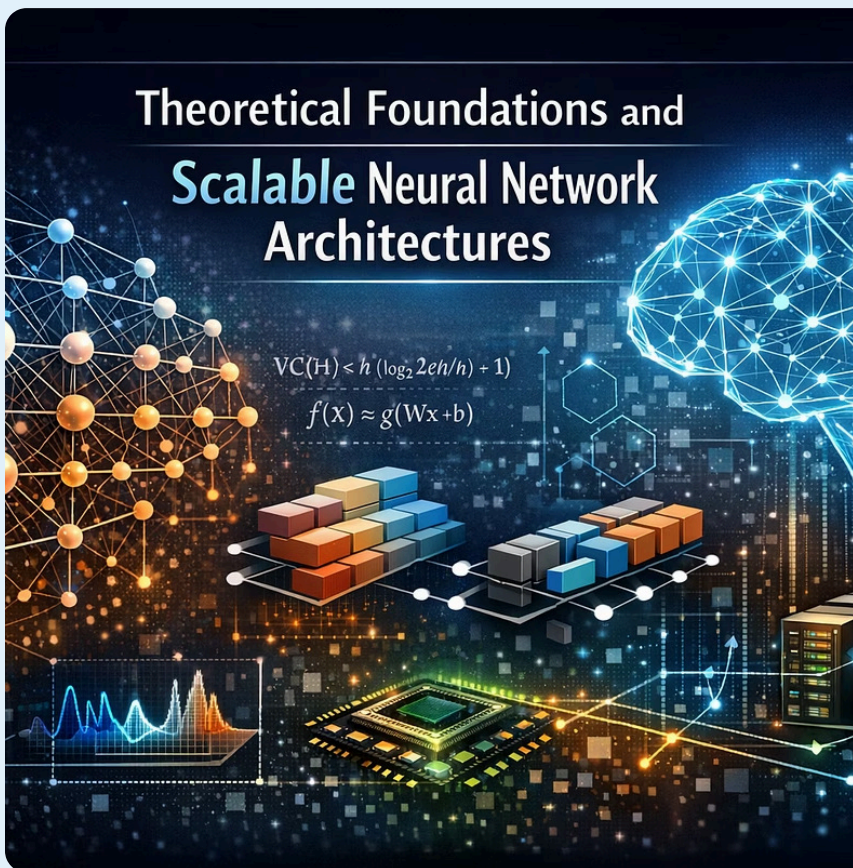


neuron

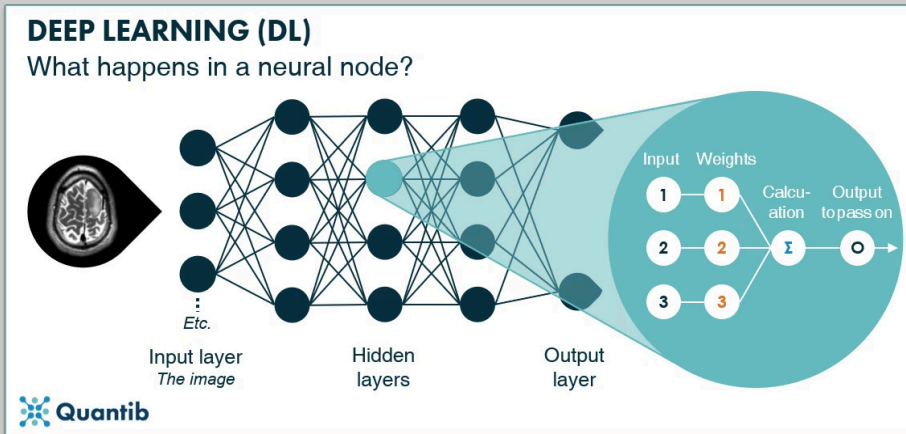


Module I: Foundations of Deep Learning Architectures

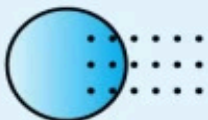
Input
(features)

Hidden Layers
lots of layers ~ “deep learning”

What is Deep Learning?



- Deep learning is a subset of machine learning driven by multilayered neural networks whose design is inspired by the structure of the human brain.
- Deep learning models can recognize complex pictures, text, sounds, and other data patterns to produce accurate insights and predictions. You can use deep learning methods to automate tasks that typically require human intelligence, such as describing images or transcribing a sound file into text.



neuron

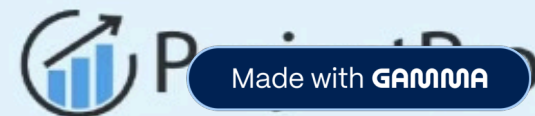


Introduction to Deep Learning Architectures

Deep learning architectures have evolved significantly over time, driven by advancements in research, computational power, and data availability. Initially, simple architectures like perceptrons paved the way for more complex models such as [convolutional neural networks \(CNNs\)](#) and [recurrent neural networks \(RNNs\)](#). Introducing dropout regularization, batch normalization, and attention mechanisms improved model performance. Recent trends include the rise of Transformer models for [natural language processing \(NLP\) systems](#) and the exploration of hybrid architectures combining neural networks with symbolic reasoning

Input
(features)

Hidden Layers
lots of layers ~ “deep learning”



Made with GAMMA

Key Components of Deep Learning Architectures

1 Neurons:



Neurons are at the heart of any neural network, including the perceptron. These digital entities receive inputs, apply weights, and produce an output. Neurons are the building blocks through which information flows in a neural network. Just as neurons in our brains communicate, these fundamental blocks communicate in the language of numbers.

2 Activation Functions :



Activation Functions such as the step function in the perceptron, determine whether the neuron fires. In other words, they decide whether the information flowing through should be allowed to contribute to the output. Picture it as a threshold—if the incoming data is above a certain level, the perceptron 'fires' or produces an output; otherwise, it remains silent. This binary decision-making process showcases the essence of activation functions in shaping the output of our digital neurons.

3 Weights and Biases:



Weights and Biases are the adjustable parameters in the network that influence the importance of input features and establish a threshold for activation. Each input data feature is given a certain weight, indicating its importance in the decision. Biases act as the minimum requirement for a feature to contribute. Adjusting weights and biases during model training refines its ability to make accurate predictions.

4 Loss Functions:



Loss Functions quantify the difference between the predicted output and the actual target. The objective is to minimize this difference during the training process. In the context of the perceptron, the concept of error or loss becomes evident. The perceptron learns by reducing the difference between its prediction and the actual target, laying the groundwork for understanding more sophisticated loss functions in advanced neural networks. Mean Squared Error (MSE) and Cross-Entropy Loss are examples of loss functions commonly used in different use cases.

5 Optimizers:



Although more straightforward in the perceptron context, [optimizers](#) are crucial for adjusting weights and biases based on the computed loss. They fine-tune the model parameters to minimize the loss and improve overall performance. This mechanism hints at the broader optimization techniques in more complex deep learning architectures. Popular optimizing techniques include Stochastic Gradient Descent (SGD), [Adam](#), and RMSprop.

Understanding these foundational deep learning concepts will help us explore more complex deep learning architectures, where multiple layers of neurons interact to solve intricate problems. Now that you are thoroughly familiar with the basics, let's explore the different types of deep learning architectures.

Universal Approximation Theorem

The **Universal Approximation Theorem** is a pivotal result in neural network theory, proving that feedforward neural networks can approximate any continuous function under certain conditions. This theorem provides a mathematical foundation for why neural networks are capable of solving complex problems across various domains like image recognition, natural language processing, and more.

How Neural Networks Approximate Functions?

Neural networks approximate functions by adjusting the weights and biases of their neurons. When a neural network is trained, it iteratively adjusts these parameters to minimize the error between its predictions and the actual outputs.

Layers and Neurons

- **Input Layer:** Accepts input data.
- **Hidden Layers:** Processes the input through weighted connections and activation functions.
- **Output Layer:** Produces the final result or prediction.

Mathematical Foundations of Function Approximation

Neural Network Structure

A [neural network](#)'s function can be described mathematically as a composition of linear transformations and activation functions.

For a network with a single hidden layer, the output is given by:

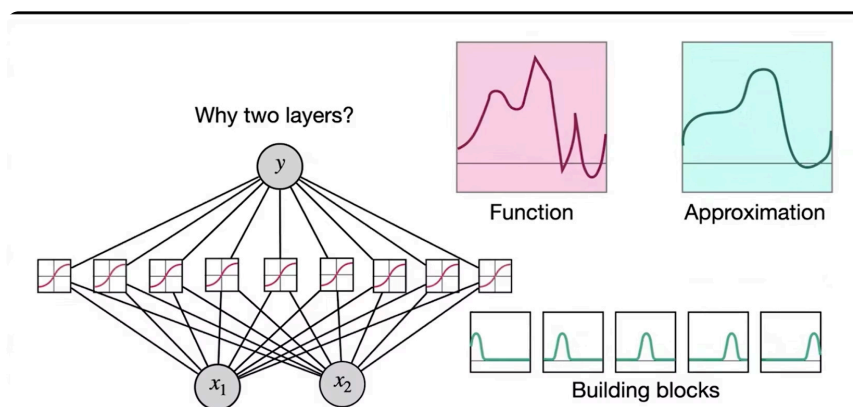
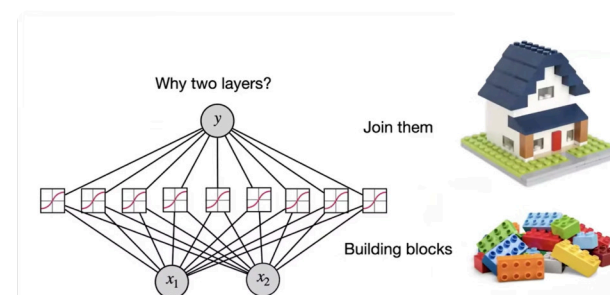
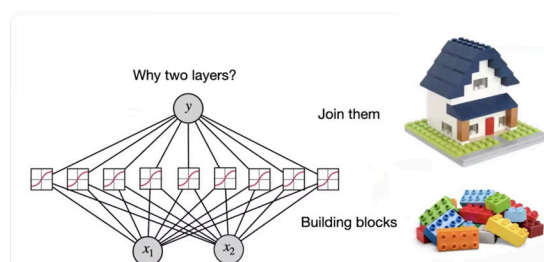
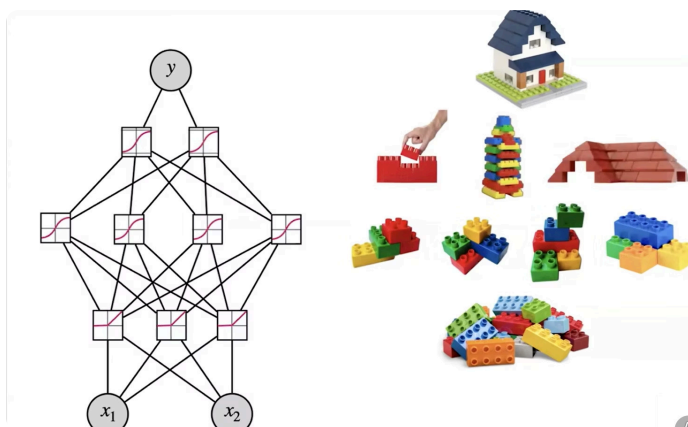
$$f'(x) = \sum_{i=1}^M c_i \cdot \sigma(W_i^T x + b_i)$$

Where:

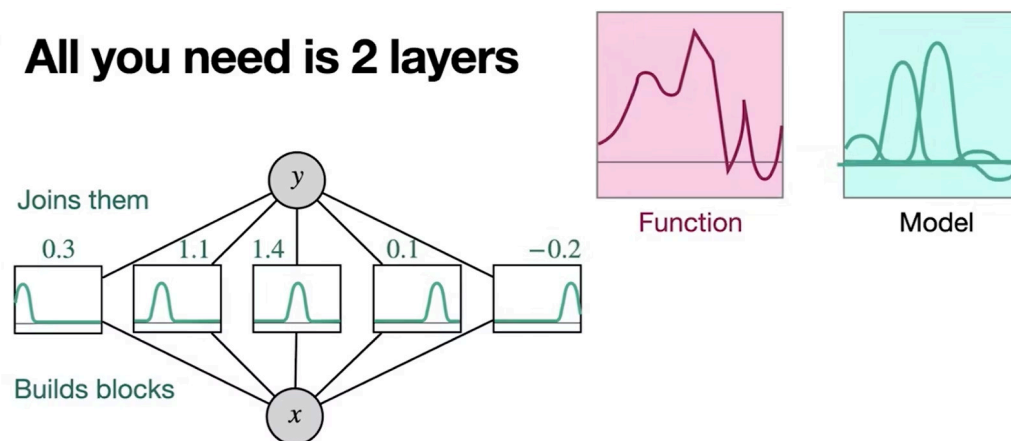
- M is the number of neurons in the hidden layer.
- c_i are the weights associated with the output layer.
- w_i and b_i are the weights and biases of the hidden neurons.
- σ is the activation function (commonly non-linear).

The idea is that, by adjusting the weights c_i , w_i and b_i , the neural network can approximate any continuous function $f(x)$ over a given domain.

How it works?(Visually)



All you need is 2 layers



Practical Limitations

While the Universal Approximation Theorem is mathematically elegant, it comes with certain practical limitations:

1

Network Size and Efficiency

The theorem guarantees that a neural network can approximate any continuous function, but it doesn't specify the size of the network required. In some cases, the number of neurons required to achieve a high degree of accuracy may be impractically large.

2

Overfitting

The network may fit the training data perfectly, but it can also overfit, meaning it performs poorly on unseen data. Regularization techniques like dropout or early stopping are needed to mitigate this risk.

3

Generalization

The theorem applies to the function approximation on the given training set, but it doesn't guarantee how well the network will generalize to new, unseen data. Techniques such as cross-validation help ensure good generalization.

4

Training Difficulties

While the theorem assures that such an approximation exists, it doesn't provide insights on how to efficiently train the network. Gradient-based optimization methods can get stuck in local minima or saddle points, making it challenging to find the optimal solution.

Vapnik-Chervonenkis (VC) Bounds

- **VC bounds** are a family of probabilistic inequalities in statistical learning theory that provide **uniform convergence guarantees** for empirical risk minimization. They bound the difference between the expected risk (true error) and empirical risk (training error) for all functions in a hypothesis class, with high probability over random training samples.
- **Vapnik–Chervonenkis (VC) Bounds** come from **statistical learning theory** and help us understand **how well a machine learning model will generalize** from training data to unseen data.

1. What is VC Dimension?

The VC dimension is a measure of a model's capacity or complexity.

It is the maximum number of data points that a model can classify correctly in all possible ways.

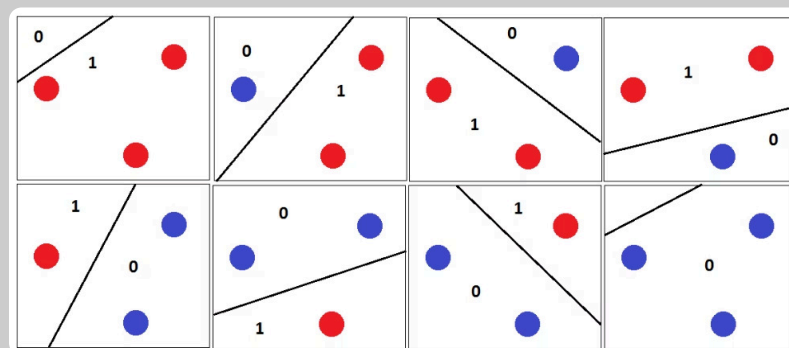
Higher VC dimension → more complex model

Lower VC dimension → simpler model

Example:

A linear classifier in 2D has VC dimension = 3

A decision tree with more depth has higher VC dimension



1. What are VC Bounds?

VC bounds give a theoretical limit on the difference between:

Training error

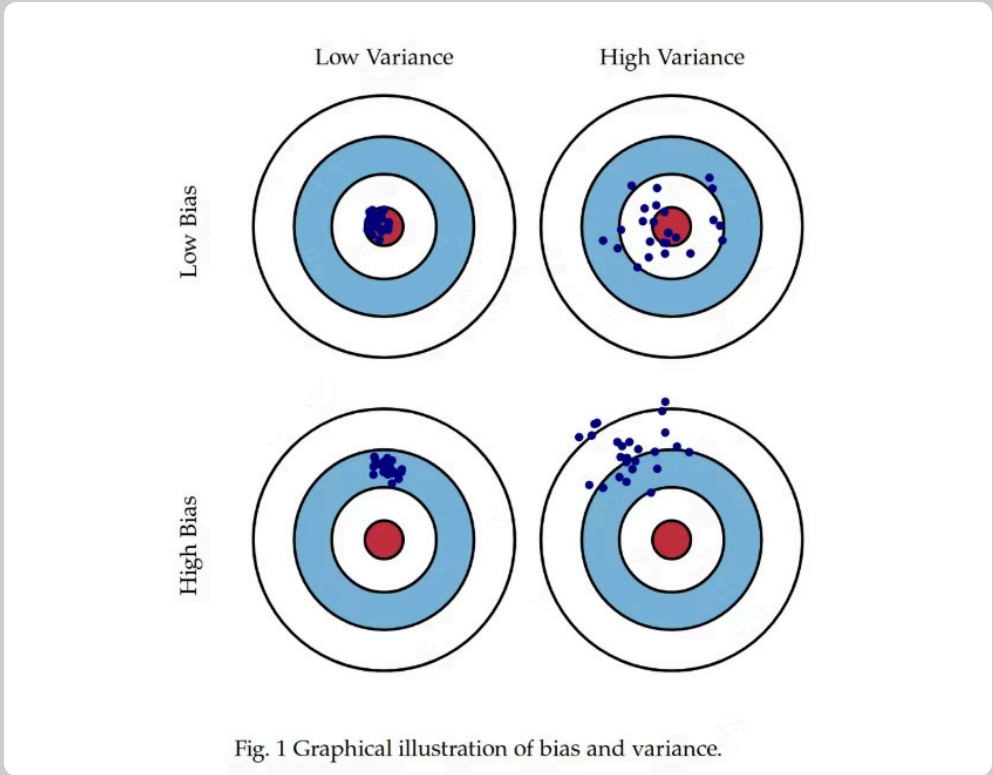
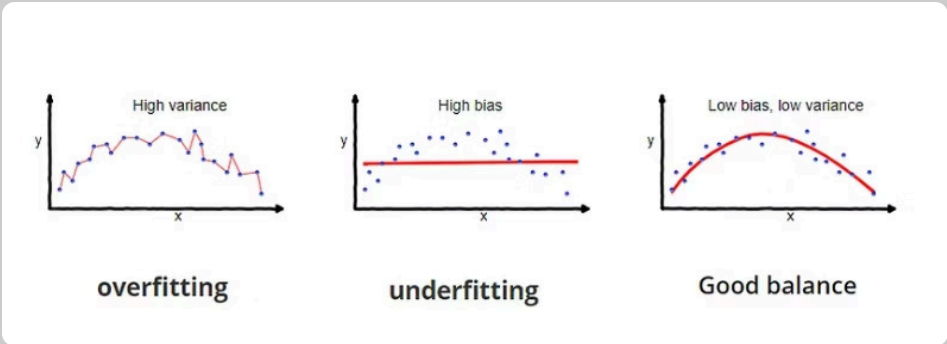
True (generalization) error

They answer:

“How much can we trust the model's performance on unseen data?”

Bias–Variance trade-off and Generalization

- **Bias:** Bias in machine learning refers to **systematic errors in model predictions resulting from faulty assumptions, unrepresentative training data, or skewed algorithms, leading to inaccurate outputs or discriminatory outcomes.**
- Bias is the difference between the average prediction of our model and the correct value which we are trying to predict. Model with high bias pays very little attention to the training data and oversimplifies the model. It always leads to high error on training and test data.
- **Variance measures how much a model's predictions change with different training datasets,** indicating its sensitivity to specific data points.
- **Variance** is the variability of model prediction for a given data point or a value which tells us spread of our data. Model with high variance pays a lot of attention to training data and does not generalize on the data which it hasn't seen before.
- **Bias (Underfitting):** Occurs when a model is too simple to capture the underlying pattern of the data. High-bias models make strong, incorrect assumptions, resulting in high error on both training and test sets (e.g., linear regression on non-linear data).
- **Variance (Overfitting):** Occurs when a model is too complex and learns noise in the training data rather than just the signal. High-variance models are overly sensitive to small fluctuations in the training set, resulting in low training error but high test error.



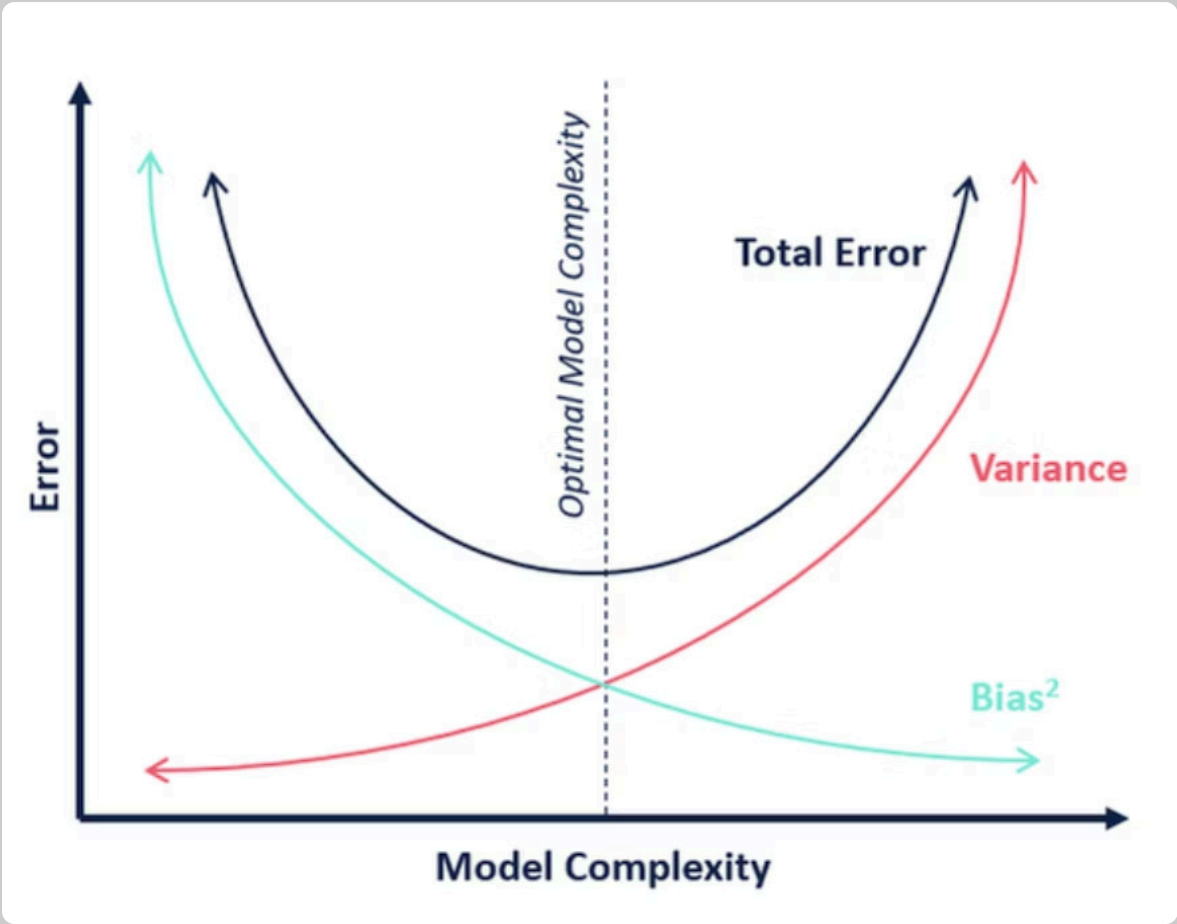
The Trade-off Mechanism

As you increase a model's complexity (e.g., adding more parameters), **bias decreases** because the model becomes flexible enough to represent the underlying data. However, **variance increases** because the model starts reacting to small variations and noise in the specific dataset used.

The total expected error of a model is decomposed as:

Total Error = Bias² + Variance + Irreducible Error

- **Irreducible Error:** Noise inherent in the data itself that no model can eliminate.
- **Sweet Spot:** The goal is to find the "sweet spot" in model complexity where the sum of bias and variance is at its minimum, resulting in the best performance on new, unseen data.



- The bias–variance trade-off is the balance between underfitting and overfitting, where decreasing bias increases variance and vice versa, and the objective is to minimize total prediction error.
- The [bias-variance tradeoff](#) refers to the balance that is needed between bias and variance to build a model that can generalize well to new data. A model that is too simple will have high bias but low variance, while a model that is too complex will have low bias but high variance. The goal is to find the right level of complexity that minimizes both bias and variance, resulting in a model that can accurately generalize to new data.
- To strike a balance between bias and variance, we want to find the optimal level of model complexity that allows it to accurately predict the prices of the houses while also generalizing well to new data. This can be done by creating an error vs. model complexity graph, which shows the performance of the model at different levels of complexity. By analyzing this graph, we can identify the optimal level of model complexity where the bias and variance trade-off is balanced.

Generalization:

Generalization is the ability of a model to perform well on new data. A model that generalizes well is able to make accurate predictions on new data, which is important if we want to use the model in the real world. On the other hand, a model that doesn't generalize well may perform well on the data that it was trained on but may not make accurate predictions on new data. This is a problem because it means the model may not be useful in practice.

When we train a machine learning model, we want it to be able to make accurate predictions not just on the data that we use to train it but on new data that it has never seen before. This is because, in the real world, we don't always have access to the exact same data that we used to train the model but new first-time-seen data points. Therefore, it's important to train models that not only fit the training data well but also generalize well to new data.

Vanishing and exploding gradient problems

Training deep [neural networks](#) often encounters challenges like the vanishing gradient and exploding gradient problems. These issues can hinder learning by either shrinking gradients to near-zero or causing them to grow uncontrollably.

What is the Vanishing Gradient?

As the **backpropagation algorithm** advances downwards(or backward) from the output layer towards the input layer, the gradients often get smaller and smaller and approach zero, eventually leaving the weights of the initial or lower layers nearly unchanged. As a result, the gradient descent never converges to the optimum. This is known as the vanishing gradients problem.

What is an Exploding Gradient?

On the contrary, the **gradients** keep getting larger in some cases as the backpropagation algorithm progresses. This, in turn, causes large weight updates and causes the gradient descent to diverge. This is known as the exploding gradient problem.

Causes

Key Causes of Vanishing Gradients

- **Saturated Activation Functions:** Functions like Sigmoid or Tanh have small derivatives (less than 1) that shrink gradients as they pass through layers.
- **Deep Network Depth:** Multiplying many small derivatives (<1) through numerous layers causes the gradient to decrease exponentially, often seen in RNNs.
- **Poor Weight Initialization:** Weights set too small result in weak signals that disappear before reaching earlier layers.

Key Causes of Exploding Gradients:

- **Large Weight Initialization:** Initializing weights to be very large causes gradients to grow exponentially as they are multiplied through layers.
- **Deep Networks & Backpropagation:** Similar to vanishing gradients, multiplying large weights or derivatives (>1) across many layers leads to exploding gradients.
- **High Learning Rate:** A learning rate that is too high can amplify updates, causing gradients to become too large and causing the model to diverge.

Solutions:

Activation Functions: Using ReLU (Rectified Linear Unit) helps prevent saturation.

Normalization: Using Batch Normalization stabilizes training.

1

Weight Initialization: Utilizing techniques like Xavier or He initialization.

2

3

Gradient Clipping: Capping gradients during training prevents them from getting too large.

4

Weight Initialization

Weight initialization in deep learning is about choosing the starting values of a neural network's weights *before training begins*. It sounds small, but it has a **huge impact** on how fast (and how well) your model learns. These weights help the network learn by adjusting them through each iteration. Poor initialization can lead to issues like:

Vanishing Gradient Problem:
Gradients become very small, slowing down or stopping training.

Exploding Gradient Problem:
Gradients become excessively large, leading to instability.

Slow Convergence: Training takes longer to reach an optimal solution.

Weight Initialization Techniques

Zero Initialization: As the name suggests, all the weights are assigned zero as the initial value is zero initialization. This kind of initialization is highly ineffective as neurons learn the same feature during each iteration. Rather, during any kind of constant initialization, the same issue happens to occur. Thus, constant initializations are not preferred.

Xavier/Glorot Initialization

Xavier initialization (also called Glorot initialization) aims to keep the variance of the activations constant across layers. This method assigns weights from a normal distribution with mean zero and a specific variance based on the number of neurons in each layer. Xavier initialization is particularly useful when using the sigmoid or tanh activation functions.

Random Initialization: In an attempt to overcome the shortcomings of Zero or Constant Initialization, random initialization assigns random values except for zeros as weights to neuron paths. However, assigning values randomly to the weights, problems such as Overfitting, Vanishing Gradient Problem, Exploding Gradient Problem might occur.

Random Initialization can be of two kinds:

- Random Normal
- Random Uniform

a) Random Normal: The weights are initialized from values in a normal distribution.

b) Random Uniform: The weights are initialized from values in a uniform distribution.

He Initialization

He initialization is similar to Xavier but better suited for layers with ReLU activations. It scales the weights by the square root of the number of incoming neurons multiplied by 2, helping prevent the exploding gradient issue and ensuring that layers using ReLU do not “die out.”

Orthogonal Initialization

Initializes the weights as an orthogonal matrix. Useful for training very deep networks. Can be used to help with vanishing and exploding gradients in RNNs.

Comparing Activation Functions with Different Initializations

Different activation functions like ReLU, tanh, and sigmoid behave differently depending on the weight initialization:

- **Sigmoid Activation:** Sigmoid can suffer from vanishing gradients, especially when initialized with very small or large weights, leading to very slow training.
- **Tanh Activation:** Similar to sigmoid, tanh can face vanishing gradients if weights are initialized too small. Xavier initialization works well here.
- **ReLU Activation:** ReLU is a non-saturating function, which helps avoid vanishing gradients. However, large values in initialization can cause exploding gradients. He initialization is typically preferred for ReLU.

Activation Functions

ReLU:

The Rectified Linear Unit (ReLU) is a popular activation function predominantly used in **deep learning** models. The function outputs a value that is directly proportional to the input, which makes it a simple yet effective utility for neural networks.

$f(x)=\max(0,x)$

▼ Functionality and Features

The ReLU function operates by outputting the input directly if it is positive; otherwise, it outputs zero. It has become the default activation function for many types of neural networks because it overcomes the vanishing gradient problem, allowing models to learn faster and perform better.

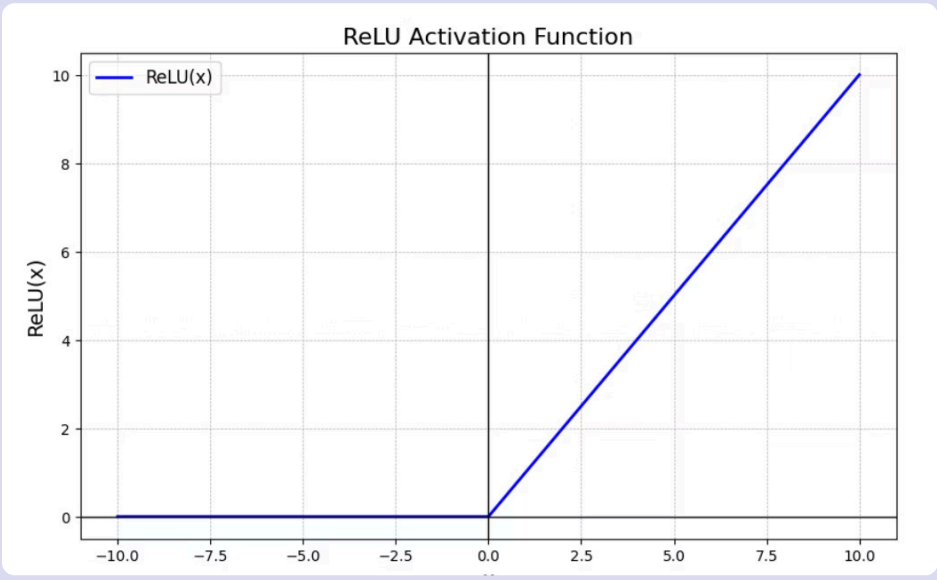
Benefits and Use Cases

The ReLU Activation Function offers several advantages, including:

- **Simplicity:** As a linear function, ReLU is easy to implement and computationally efficient.
- **Solving the Vanishing Gradient Problem:** ReLU helps to mitigate the vanishing gradient problem in neural networks, enabling them to learn from the backpropagation process more effectively.
- **Sparsity:** ReLU activation function results in a sparse representation which is beneficial in terms of memory storage and computation.

Challenges and Limitations

Despite its advantages, ReLU comes with certain limitations. The most significant one is the "Dead ReLU" problem, where neurons can sometimes become stuck during the learning process and cease to update, making them unresponsive to variations in error.



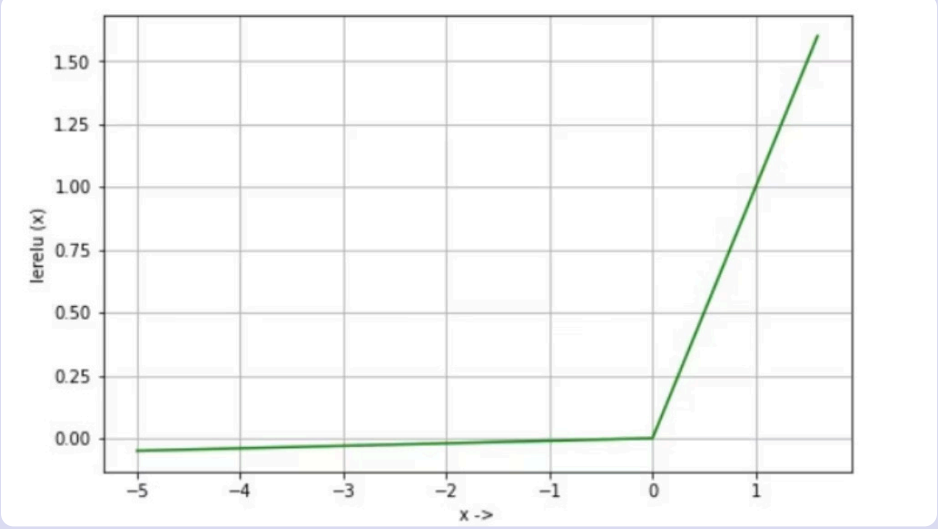
Leaky ReLu:

To mitigate the dying ReLU problem, Leaky ReLU introduces a small gradient for negative inputs, preserving some activity in the neurons. However, it struggles with consistency for negative inputs, using a fixed slope throughout training.

Instead of returning zero for negative inputs it allows a small, non-zero value. It introduces a slight modification to the standard ReLU by assigning a small, fixed slope to the negative part of the input. This ensures that neurons don't become inactive during training as they can still pass small gradients even when receiving negative values.

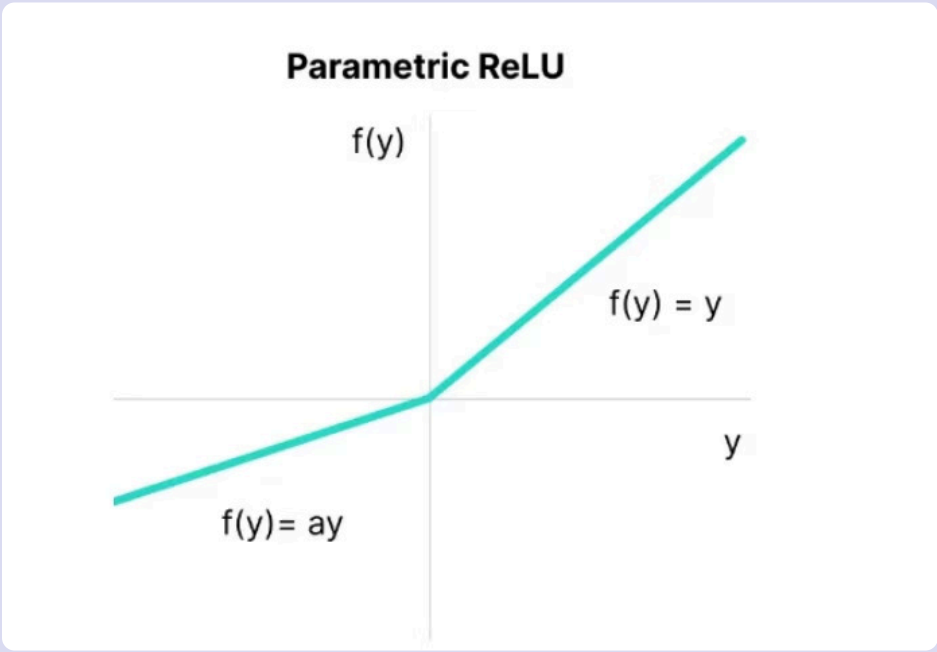
Its equation is:

$$\text{Leaky ReLU}(x) = \begin{cases} x, & \text{if } x > 0 \\ 0.01 \cdot x, & \text{if } x \leq 0 \end{cases}$$



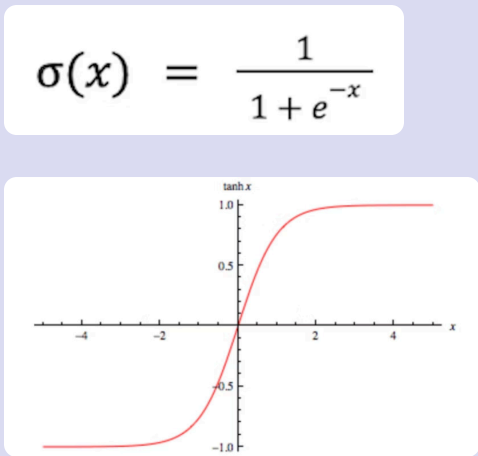
Parametric ReLu

Parametric ReLU (PReLU) is an activation function,that improves upon standard ReLU by introducing a learnable parameter a to define the slope for negative input values. It solves the "dying ReLU" problem by allowing, rather than forcing, negative inputs to have a small, optimized gradient, improving performance in deep learning models.



Activation Functions

Sigmoid Function :The **sigmoid function**, specifically the logistic function, is a fundamental mathematical tool in machine learning **used primarily to map any real-valued number to a value between 0 and 1**. This S-shaped curve function is crucial for introducing non-linearity into models and converting raw outputs into interpretable probabilities.



Swish

The Swish function, or the self-gated function, is just another activation function proposed by **Google**. It has been proposed as a possible improvisation to the already existing sigmoid function.

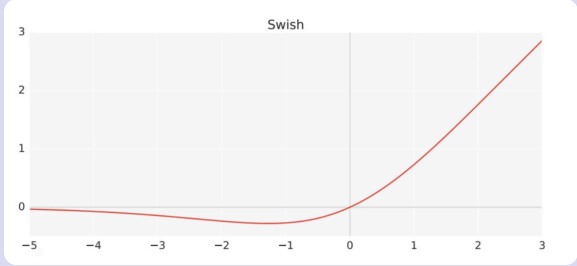
Swish(x)=x⋅sigmoid(βx), where β is a scalable and trainable parameter.

This function, unlike the ReLU function is not outputting 0 for all the negative values while it also maintains it's characteristics for the positive values of x.

Application of Swish Activation Functions in Real life scenario

Swish function is a powerful activation function which is rather useful in classification tasks. Some applications of Swish activation function are:

1. In case of Image Classification (as also researched by Google), swish activation function performs better in ImageNet and other Artificial Neural Networks which suffer from vanishing gradient issues.
2. Swish functions smooth gradient properties can help RNN learn long range dependencies which will be useful in text classification and Sentiment analysis task.
3. This function can be applied to both Generator and Discriminator model in GANs, helping in generating better samples



Tanh

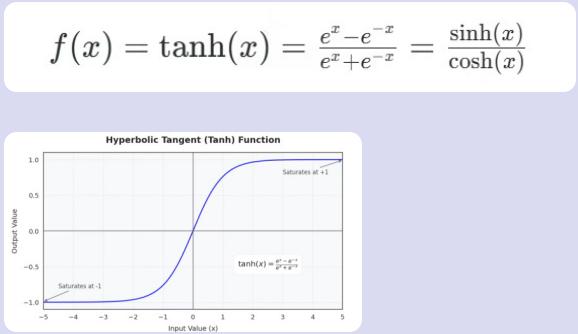
Tanh (hyperbolic tangent) is a type of activation function that transforms its input into a value between -1 and 1.

When to Use Tanh?

The tanh function is useful when:

- The data is already normalized or centered around zero.
- You are building shallow neural networks (i.e., networks with fewer layers).
- You are working with data where negative values are significant and should be retained.

It is mathematically defined as:



Mish

Mish is a smooth, non-monotonic, and self-regularized activation function proposed by [Diganta Misra](#) in 2019. It has gained popularity for its ability to outperform standard activation functions like [ReLU](#) and Swish in deep neural network architectures.

1. Mathematical Formula

Mish is defined as a gated combination of the identity function, the hyperbolic tangent ([tanh](#)), and the softplus function:

$$\text{Mish}(x) = x \cdot \tanh(\text{softplus}(x)) = x \cdot \tanh(\ln(1 + e^x))$$

Key Properties

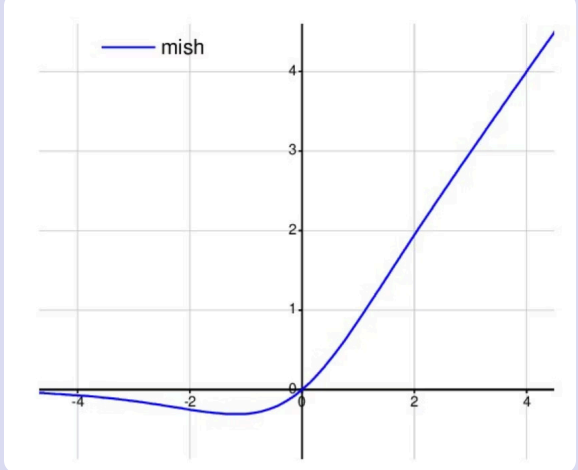
- **Non-Monotonicity:** Unlike ReLU, Mish has a small negative dip (minimum at $x \approx -1.1924$ with value ≈ -0.30884) which helps preserve small negative values and improves gradient flow.

Unbounded Above: Similar to ReLU, it can extend to infinity for positive inputs, preventing the saturation problem where gradients become nearly zero.

- **Bounded Below:** It is bounded at approximately -0.31, providing a regularizing effect that helps in better generalization.
- **Continuity & Smoothness:** Mish is continuous (infinitely differentiable), meaning it has no sharp transitions like the "elbow" of ReLU. This results in smoother loss landscapes that are easier to optimize.

Performance & Usage

- **Benchmarks:** Mish consistently outperforms ReLU and Swish in tasks like image classification (CIFAR-10/100, ImageNet) and object detection (MS-COCO). Notably, it was a key component in the [YOLOv4](#) architecture.
- **Computational Cost:** It is more mathematically complex and thus slightly slower than ReLU. To mitigate this, developers often use optimized versions like Mish-CUDA for faster training.
- **Implementation:** It is natively supported in major deep learning frameworks:
 - **PyTorch:** Available as [torch.nn.Mish](#).
 - **TensorFlow:** Accessible via [tf.keras.activations.mish](#)



Softmax

The softmax function is a function that turns a vector of **K** real values into a vector of **K** real values that sum to 1. The input values can be positive, negative, zero, or greater than one, but the softmax transforms them into values between 0 and 1, so that they can be interpreted as probabilities. If one of the inputs is small or negative, the softmax turns it into a small probability, and if an input is large, then it turns it into a large probability, but it will always remain between 0 and 1.

Softmax is a generalization of logistic regression that can be used for multi-class classification, and its formula is very similar to the sigmoid function which is used for logistic regression. The softmax function can be used in a classifier only when the classes are mutually exclusive.

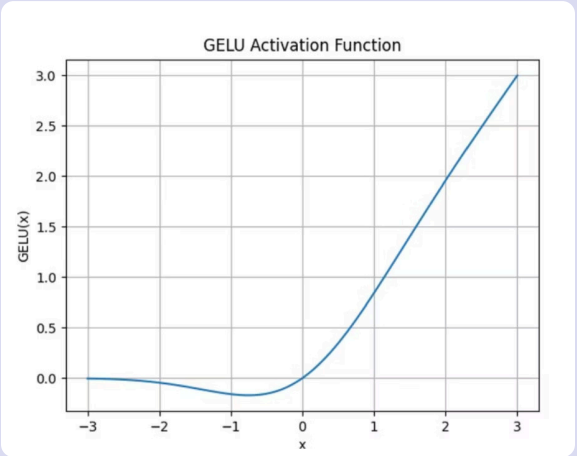
GeLU

The Gaussian Error Linear Unit (GELU) is a sophisticated [activation function](#) that plays a pivotal role in the performance of modern [artificial intelligence \(AI\)](#) systems, particularly those based on the Transformer architecture. Unlike traditional functions that apply a rigid, deterministic threshold to neuron inputs, GELU introduces a probabilistic aspect inspired by the properties of the [Gaussian distribution](#). By weighing inputs by their magnitude rather than simply gating them, GELU provides a smoother nonlinearity that aids in the optimization of [deep learning \(DL\)](#) models. This unique characteristic allows networks to model complex data patterns more effectively, contributing significantly to the success of massive [foundation models](#).

How GELU Works

At the core of any [neural network](#), activation functions determine whether a neuron "fires" based on its input signal. Older functions like the [Rectified Linear Unit \(ReLU\)](#) operate like a switch, outputting zero for any negative input and the input itself for positive values. While efficient, this sharp cutoff can hinder training dynamics.

GELU improves upon this by scaling the input by the cumulative distribution function of a Gaussian distribution. Intuitively, this means that as the input value decreases, the probability of the neuron dropping out increases, but it happens gradually rather than abruptly. This curvature creates a smooth, non-monotonic function that is differentiable at all points. This smoothness facilitates better [backpropagation](#) of gradients, helping to mitigate issues like the [vanishing gradient problem](#) which can stall the training of deep networks.



Comparison of Common Activation Functions

Activation Function	Output Range	Key Characteristics	Advantages	Limitations	Typical Use Cases
ReLU	$[0, \infty)$	Piecewise linear, non-saturating for positive inputs	Fast computation, mitigates vanishing gradient, sparse activation	Dead ReLU problem, unbounded output	Hidden layers in CNNs & DNNs
Leaky ReLU	$(-\infty, \infty)$	Small slope for negative inputs	Reduces dead ReLU problem, simple	Fixed negative slope may be suboptimal	Deep CNNs
PReLU	$(-\infty, \infty)$	Learnable negative slope	Adaptive, better gradient flow	Risk of overfitting, more parameters	Very deep networks
Sigmoid	$(0, 1)$	S-shaped, probabilistic output	Interpretable as probability	Vanishing gradients, slow convergence	Binary classification (output layer)
Tanh	$(-1, 1)$	Zero-centered sigmoid	Stronger gradients than sigmoid	Still suffers from vanishing gradients	Shallow networks, RNNs
Softmax	$(0, 1)$, sum = 1	Probability distribution across classes	Multi-class probability output	Sensitive to large inputs	Multi-class classification (output layer)
Swish	$(-\infty, \infty)$	Smooth, non-monotonic	Better gradient flow, outperforms ReLU	Higher computational cost	CNNs, RNNs, GANs
Mish	$\approx [-0.31, \infty)$	Smooth, non-monotonic, self-regularized	Strong generalization, smooth loss	Computationally expensive	Object detection, YOLOv4
GELU	$(-\infty, \infty)$	Probabilistic gating, smooth	Excellent optimization behavior	Slower than ReLU	Transformers (BERT, GPT, ViT)

Normalization Techniques

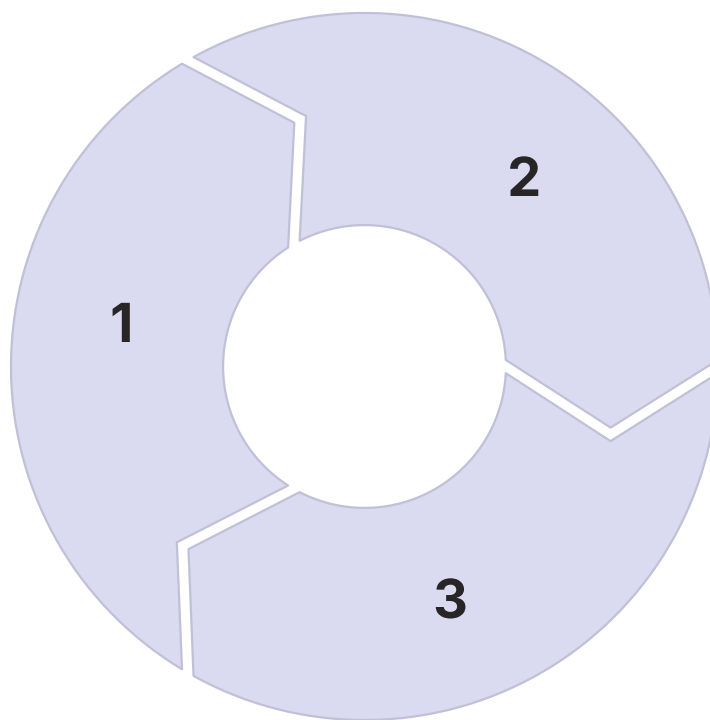
Normalization:

Normalization, aka **Feature Scaling**, is a method used to normalize the range of independent variables or features of data. It is generally performed during the data preprocessing step.

Normalization in deep learning is a **preprocessing and architectural technique that rescales input data or layer activations to a standard range (e.g., 0-1) or distribution (mean 0, variance 1)**

Batch Normalization:

Batch Normalization (BN) is perhaps one of the most widely used normalization techniques in deep learning. Introduced by Sergey Ioffe and Christian Szegedy in 2015, BN operates by normalizing the activations of each layer across the mini-batch during training. This ensures that the distribution of inputs remains stable throughout the network, leading to faster convergence and improved generalization. BN also introduces learnable parameters, allowing the model to adaptively scale and shift the normalized activations, further enhancing its flexibility and performance.



Layer Normalization

Layer normalization is a technique used in deep learning to stabilize the training of neural networks. It works by normalizing the inputs across the features for each training example. This contrasts with batch normalization, which normalizes across the batch dimension (i.e., different training examples). Layer normalization is particularly useful in recurrent neural networks (RNNs) and has also been successfully applied in transformers and other network architectures.

RMS Norm

Root Mean Square Normalization (RMSNorm) is a relatively novel normalization technique introduced by Biao Zhang, Rico Sennrich in 2019. Unlike BN and LN, RMSNorm normalizes activations based on the root mean square of the activations themselves, rather than using mini-batch or layer statistics. This approach ensures that the activations are consistently scaled regardless of the mini-batch size or the number of features. Additionally, RMSNorm introduces learnable scale parameters, offering similar adaptability to BN.

Regularization & Its Strategies

Regularization is a technique used in machine learning and deep learning to prevent overfitting and improve a model's generalization performance. It involves adding a penalty term to the [loss function](#) during training.

This penalty discourages the model from becoming too complex or having large parameter values, which helps in controlling the model's ability to fit noise in the training data. Regularization in deep learning [methods](#) includes L1 and L2 regularization, dropout, early stopping.

L2 & L1 Regularization

L1 and L2 are the most common types of regularization deep learning. These update the general cost function by adding another term known as the regularization term.

- *Cost function = Loss (say, binary cross entropy) + Regularization term*

Due to the addition of this regularization term, the values of weight matrices decrease because it assumes that a neural network with smaller weight matrices leads to simpler models. Therefore, it will also reduce overfitting to quite an extent.

However, this regularization term differs in L1 and L2.

For L2:

$$\text{Cost function} = \text{Loss} + \frac{\lambda}{2m} * \sum \|w\|^2$$

For L1:

$$\text{Cost function} = \text{Loss} + \frac{\lambda}{2m} * \sum \|w\|$$

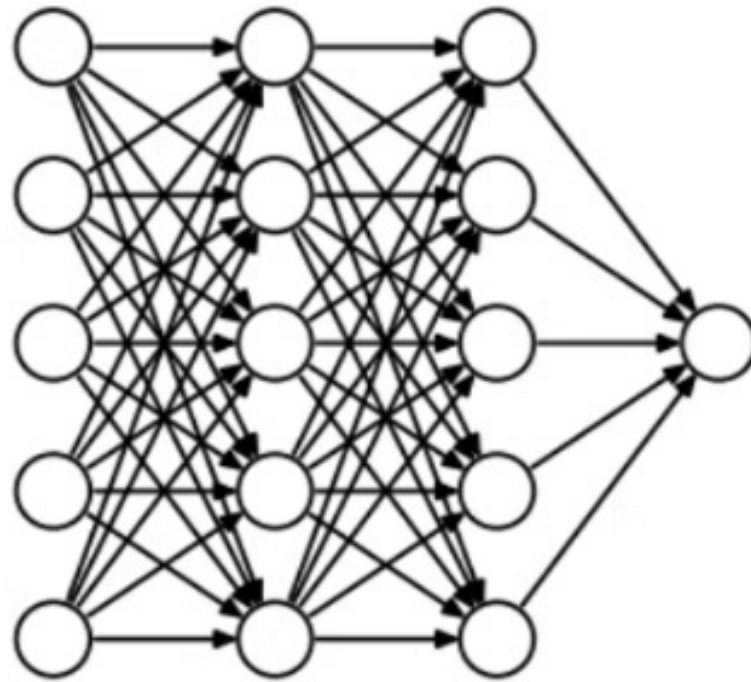
In L2, we have: $\|w\|^2 = \sum w_i^2$. This is known as ridge regression, where lambda is the regularization parameter. It is the hyperparameter whose value is optimized for better results. L2 regularization is also known as weight decay as it forces the weights to decay towards zero (but not exactly zero).

In L1, we having: $\|w\| = \sum |w_i|$. In this, we penalize the absolute value of the weights. Unlike L2, the weights may be reduced to zero here. L1 regularization is also called lasso regression. **Hence, it is very useful when we are trying to compress our model. Otherwise, we usually prefer L2 over it.**

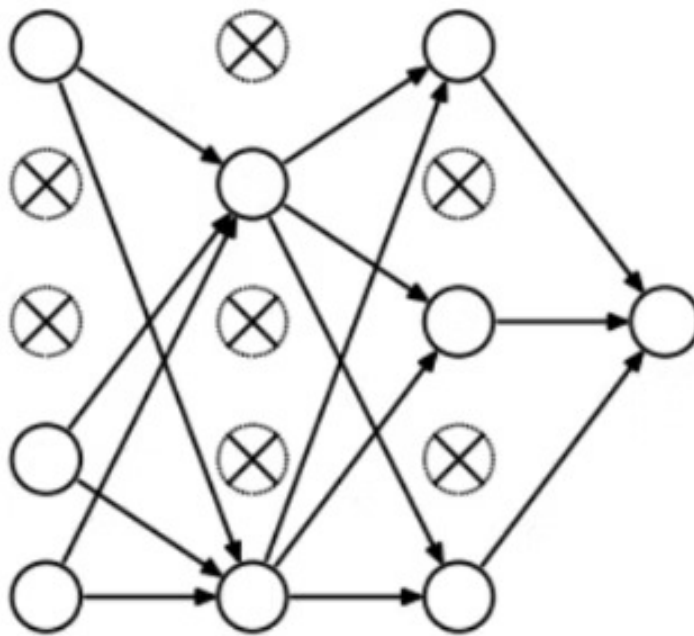
Dropout

This is one of the most interesting types of [regularization techniques](#). It also produces very good results and is consequently the most frequently used regularization technique in the field of deep learning.

To understand dropout, let's say our neural network structure is akin to the one shown below:



So what does dropout do? At every iteration, it randomly selects some nodes and removes them along with all of their incoming and outgoing connections as shown below:



Each iteration has a different set of nodes, which results in a different set of outputs. This can also be thought of as an ensemble technique in machine learning.

Ensemble models usually perform better than a single model as they capture more randomness. Similarly, dropout models also perform better than normal neural network models.

This probability of choosing how many nodes should be dropped is the hyperparameter of the dropout function. As seen in the image above, dropout can be applied to both the hidden layers as well as the input layers.

Data Augmentation

The simplest way to reduce overfitting is to increase the training data size. In machine learning, however, increasing the training data size was impossible as the labeled data was too costly.

But now, let's consider we are dealing with images. In this case, there are a few ways of increasing the size of the training data—rotating the image, flipping, scaling, shifting, etc. In the image below, some transformation has been done on the handwritten digits dataset.

shift



shift



shear



shift & scale



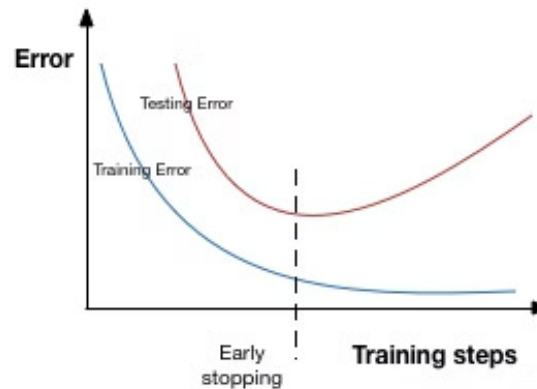
rotate & scale



This technique is known as data augmentation. It usually provides a big leap in improving the accuracy of the model, and it can be considered a mandatory trick to improve our predictions.

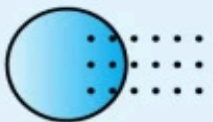
Early Stopping

Early stopping is a [cross-validation](#) strategy in which we keep one part of the training set as the validation set. When we see that the performance on the validation set is getting worse, we immediately stop the training on the model.



In the above image, we will stop training at the dotted line since, after that, our model will start overfitting on the training data.

Hidden Layers
lots of layers ~ “deep learning”



neuron



Label smoothing

